



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN SOFTWARE

DOCUMENTO DE ESPECIFICACIONES
NIVEL OPERATIVO - RETAILMIND SHOP

EV 09

Materia: Construcción de Software

Nombre: Benites Pérez Dariem Alberto

Sexto Semestre

Indice

- Indice..... 1
- 1. Introduccion y descripcion del sistema..... 7
- 2. Niveles empresariales y su relacion con la especificacion..... 8
- 3. Especificacion general del sistema operativo 9
 - 3.1 Objetivo general 9
 - 3.2 Actores y roles 9
 - 3.3 Modulos / paquetes operativos..... 9
 - 3.4 Reglas y restricciones generales..... 10
- 4. Especificaciones por modulo 12
 - 4.1 Seguridad y Control de Acceso 12
 - 1. Objetivo 12
 - 2. Actores 12
 - 3. Contexto..... 12
 - 4. Requisitos funcionales 12
 - 5. Requisitos no funcionales..... 13
 - 6. Reglas de negocio 13
 - 7. Entradas 13
 - 8. Salidas (incluye mensajes de error) 13
 - 9. Escenarios (Gherkin)..... 14
 - 10. Criterios de aceptacion 14
 - 11. Restricciones..... 15
 - 12. Dependencias 15
 - 13. Fuera de alcance..... 15
 - 4.2 Catalogo 16
 - 1. Objetivo 16
 - 2. Actores..... 16

3. Contexto.....	16
4. Requisitos funcionales.....	16
5. Requisitos no funcionales.....	16
6. Reglas de negocio	17
7. Entradas	17
8. Salidas (incluye mensajes de error)	17
9. Escenarios (Gherkin).....	17
10. Criterios de aceptacion	18
11. Restricciones.....	18
12. Dependencias	18
13. Fuera de alcance.....	18
4.3 Proveedores y Compras.....	19
1. Objetivo.....	19
2. Actores.....	19
3. Contexto.....	19
4. Requisitos funcionales.....	19
5. Requisitos no funcionales.....	19
6. Reglas de negocio	20
7. Entradas	20
8. Salidas (incluye mensajes de error)	20
9. Escenarios (Gherkin).....	20
10. Criterios de aceptacion	21
11. Restricciones.....	21
12. Dependencias	21
13. Fuera de alcance.....	22
4.4 Inventario y Bodegas.....	23
1. Objetivo.....	23
2. Actores.....	23
3. Contexto.....	23
4. Requisitos funcionales.....	23

5. Requisitos no funcionales.....	23
6. Reglas de negocio	23
7. Entradas	24
8. Salidas (incluye mensajes de error)	24
9. Escenarios (Gherkin).....	24
10. Criterios de aceptacion	25
11. Restricciones.....	25
12. Dependencias	25
13. Fuera de alcance.....	25
4.5 Ventas y Pedidos	26
1. Objetivo.....	26
2. Actores.....	26
3. Contexto.....	26
4. Requisitos funcionales.....	26
5. Requisitos no funcionales.....	26
6. Reglas de negocio	26
7. Entradas	27
8. Salidas (incluye mensajes de error)	27
9. Escenarios (Gherkin).....	27
10. Criterios de aceptacion	28
11. Restricciones.....	28
12. Dependencias	28
13. Fuera de alcance.....	28
4.6 Facturacion (compra y venta).....	29
1. Objetivo.....	29
2. Actores.....	29
3. Contexto.....	29
4. Requisitos funcionales.....	29
5. Requisitos no funcionales.....	29
6. Reglas de negocio	29

7. Entradas	30
8. Salidas (incluye mensajes de error)	30
9. Escenarios (Gherkin).....	30
10. Criterios de aceptacion	31
11. Restricciones.....	31
12. Dependencias	31
13. Fuera de alcance.....	31
4.7 Logistica y Envios	32
1. Objetivo.....	32
2. Actores	32
3. Contexto.....	32
4. Requisitos funcionales.....	32
5. Requisitos no funcionales.....	32
6. Reglas de negocio	32
7. Entradas	33
8. Salidas (incluye mensajes de error)	33
9. Escenarios (Gherkin).....	33
10. Criterios de aceptacion	33
11. Restricciones.....	33
12. Dependencias	34
13. Fuera de alcance.....	34
4.8 Devoluciones.....	35
1. Objetivo.....	35
2. Actores	35
3. Contexto.....	35
4. Requisitos funcionales.....	35
5. Requisitos no funcionales.....	35
6. Reglas de negocio	35
7. Entradas	36
8. Salidas (incluye mensajes de error)	36

9. Escenarios (Gherkin).....	36
10. Criterios de aceptacion	36
11. Restricciones.....	36
12. Dependencias	37
13. Fuera de alcance.....	37
5. Casos de uso del nivel operativo	38
5.1 Seguridad y Control de Acceso	38
5.2 Catalogo	38
5.3 Proveedores y Compras.....	39
5.4 Inventario y Bodegas.....	40
5.5 Ventas y Pedidos	40
5.6 Facturacion.....	41
5.7 Logistica y Envios	41
5.8 Devoluciones.....	42
6. Matriz de trazabilidad consolidada	43
7. Hallazgos y deuda tecnica	44

1. Introduccion y descripcion del sistema

RetailMind es una plataforma web de comercio electronico tipo PyME con un back-office administrativo completo. A diferencia de una tienda unicamente de cara al cliente, el sistema cubre de extremo a extremo los dos ciclos de negocio que sostienen la operacion: el abastecimiento (Procure-to-Pay) y la venta (Order-to-Cash), incluyendo inventario, bodegas, facturacion, logistica y devoluciones.

La arquitectura es hibrida y esta separada por responsabilidad de datos:

- PostgreSQL (base de datos 'retailmind', ~102 tablas transaccionales) es la fuente de verdad de todo lo operativo: usuarios, catalogo, proveedores, compras, inventario, pedidos, facturas, envios y devoluciones.
- ClickHouse se mantiene exclusivamente para analitica (nivel tactico y estrategico) sobre eventos de comportamiento; no participa del ciclo transaccional operativo.

Nota de contexto: la documentacion de steering del repositorio afirma que 'PostgreSQL fue eliminado'. Esa afirmacion esta desactualizada respecto del codigo y del esquema reales: el back-office operativo se ejecuta integramente sobre PostgreSQL (se verifico conectando a la BD y leyendo backend y frontend). Este documento refleja el sistema real actual.

Componentes del sistema:

- Backend API (Java 17 / Spring Boot 3.5): expone la operacion via REST, con acceso a datos por JdbcTemplate (sin JPA) y consultas parametrizadas. Genera PDF de facturas (iText).
- Frontend (Angular 17 standalone, lazy-loaded): SPA con las pantallas operativas agrupadas en features/operativo (catalogo, compras, inventario, ventas, horarios).
- PostgreSQL: motor operativo con seguridad a nivel de base de datos (roles de grupo, matriz de privilegios, RLS y restriccion horaria por triggers).

El presente documento especifica unicamente el NIVEL OPERATIVO (el que genera y sostiene las transacciones del negocio). Los niveles tactico y estrategico se referencian solo para ubicar el alcance.

2. Niveles empresariales y su relacion con la especificacion

El dominio se organiza en tres niveles empresariales. El nivel operativo es el foco de este documento y cubre los ciclos Procure-to-Pay (compra) y Order-to-Cash (venta).

Nivel	Proposito	Alcance en RetailMind
Operativo	Ejecuta y registra las transacciones del negocio dia a dia.	Seguridad y acceso, catalogo, compras, inventario, ventas, facturacion, logistica y devoluciones.
Tactico	Apoya la toma de decisiones de mando medio.	Sesiones, conversiones, funnel, analytics por region/dispositivo/trafico y reportes (Excel/PDF).
Estrategico	Indicadores ejecutivos de alto nivel.	Dashboard de KPIs.

Mapa de paquetes del nivel operativo y sus objetivos:

Paquete operativo	Objetivo	Ciclo
Seguridad y Control de Acceso	Autenticar, autorizar (RBAC + roles de motor) y restringir por horario.	Transversal
Catalogo	Mantener productos, variantes, categorias, marcas y atributos publicables.	Soporte a venta
Proveedores y Compras	Abastecer: ordenes de compra, recepcion y pago a proveedores.	Procure-to-Pay
Inventario y Bodegas	Controlar existencias y movimientos (kardex) entre bodegas.	Procure-to-Pay / O2C
Ventas y Pedidos	Registrar pedidos y su ciclo de estados.	Order-to-Cash
Facturacion	Emitir facturas de compra y de venta con su PDF; abrir cuentas por pagar.	P2P / O2C
Logistica y Envios	Despachar pedidos y dar seguimiento al envio.	Order-to-Cash
Devoluciones	Procesar devoluciones (RMA) y reingresar stock.	Order-to-Cash

3. Especificacion general del sistema operativo

3.1 Objetivo general

Proveer una plataforma operativa que soporte el ciclo completo de comercio de una PyME: abastecer mercancia desde proveedores, controlar el inventario en bodegas, vender al cliente, facturar, despachar y gestionar devoluciones; todo ello con seguridad aplicada tanto en la capa web (JWT + RBAC) como en la propia base de datos (roles de grupo, privilegios, RLS y restriccion horaria).

3.2 Actores y roles

El sistema define 8 roles. Cada rol de aplicacion (tabla 'rol', codigo viaja en el JWT) se mapea por lista blanca a un rol de grupo de PostgreSQL (grp_*) que la aplicacion asume por transaccion mediante SET LOCAL ROLE.

Rol (app)	Codigo	Rol de motor	Responsabilidad	Horario
Administrador	ADMIN	grp_administrador	Acceso total; administra usuarios, horarios y catalogo.	Exento (24/7)
Gerente	GERENTE	grp_gerente	Lectura total y aprobaciones.	L-V 08:00-18:00; Sab 08:00-13:00
Vendedor	VENDEDOR	grp_vendedor	Ciclo de venta (pedidos, facturacion).	L-V 08:00-18:00; Sab 08:00-13:00
Encargado de Compras	COMPRAS	grp_compras	Ciclo de abastecimiento (ordenes, facturas de compra).	L-V 08:00-18:00; Sab 08:00-13:00
Encargado de Bodega	BODEGA	grp_bodega	Inventario fisico: recepcion y transferencias.	Lun 07:00-17:00; Mar-Vie 08:00-18:00; Sab 08:00-13:00
Encargado de Despacho	DESPACHO	grp_despacho	Envios y seguimiento.	L-V 08:00-18:00; Sab 08:00-13:00
Cliente	CLIENTE	grp_cliente	Tienda en linea; solo sus propias filas (RLS).	24/7
Analista de Datos	ANALISTA	grp_analista	Solo lectura de negocio.	L-V 08:00-18:00; Sab 08:00-13:00

Los horarios provienen de la tabla grupo_horario (verificada en la BD). El administrador y cualquier superusuario quedan exentos por diseno de la funcion esta_en_horario.

3.3 Modulos / paquetes operativos

- 4.1 Seguridad y Control de Acceso
- 4.2 Catalogo
- 4.3 Proveedores y Compras
- 4.4 Inventario y Bodegas
- 4.5 Ventas y Pedidos
- 4.6 Facturacion (compra y venta)

- 4.7 Logística y Envíos
- 4.8 Devoluciones

3.4 Reglas y restricciones generales

Estas reglas son transversales a todos los módulos operativos y están implementadas en el código y/o en el esquema de PostgreSQL (verificadas contra el sistema real):

- **RN-GEN-01:** Autenticación STATELESS con JWT. El login es por email y las contraseñas se almacenan con BCrypt en `usuario.password_hash`. La autorización de rutas (RBAC) se declara en `SecurityConfig` por `authority` (ADMIN, GERENTE, VENDEDOR, COMPRAS, BODEGA, DESPACHO, CLIENTE, ANALISTA).
- **RN-GEN-02:** Seguridad de motor por transacción: el aspecto `PgSessionRoleAspect` ejecuta, como primera sentencia de cada transacción, `'SET LOCAL ROLE grp_*` según el rol del JWT. El nombre del rol solo puede salir de la lista blanca `DbGroupRole` (enum), lo que elimina la inyección SQL en el SET ROLE. Al COMMIT/ROLLBACK el rol muere y la conexión vuelve al pool como `retailmind_app` (NOINHERIT, sin privilegios de negocio).
- **RN-GEN-03:** Aislamiento por RLS: 34 tablas tienen Row-Level Security. Las políticas `'pol_horario'` cubren a los roles de staff y las `'pol_cliente_propio'` restringen a `grp_cliente` a sus propias filas mediante `fn_cliente_actual()/fn_usuario_actual()` y el parámetro de sesión `app_cliente_id`.
- **RN-GEN-04:** Restricción horaria: la función `esta_en_horario(rol)` consulta `grupo_horario` y los triggers `fn_bloquear_fuera_horario` impiden INSERT/UPDATE/DELETE fuera de la ventana del rol (error SQLSTATE 42501). El administrador/superusuario está exento; el cliente opera 24/7.
- **RN-GEN-05:** Totales por trigger: los totales de cabecera (`pedido`, `orden_compra`, `factura_compra`, `factura_venta` y `devolucion`) los recalculan triggers `fn_recalcular_total_*`; la aplicación nunca los calcula ni escribe, solo los LEE después de insertar el detalle.
- **RN-GEN-06:** Columnas generadas: `*_detalle.subtotal = cantidad * precio_unitario` es GENERATED ALWAYS (`pedido_detalle`, `orden_compra_detalle`, `factura_compra_detalle`, `factura_venta_detalle`). Jamás se inserta ni se actualiza.
- **RN-GEN-07:** Snapshots en documentos: al facturar/pedir se copian nombre, SKU y precio vigentes al detalle; `factura_venta` guarda `razon_social`, `identificacion` y `direccion_facturacion` del cliente en el momento de la emisión.
- **RN-GEN-08:** Kardex obligatorio: todo movimiento de stock pasa por `StockService`, que hace `upsert` de inventario + `SELECT ... FOR UPDATE` + fila en `movimiento_inventario` (referencia polimórfica) + `UPDATE` del stock. El signo lo fija el 'factor' del catálogo tipo `movimiento` (lista blanca). No se permite stock negativo.
- **RN-GEN-09:** `fecha_actualizacion` la mantienen triggers `*_touch` (`fn_touch_fecha_actualizacion`) en cada UPDATE.

- **RN-GEN-10:** Numeracion de folios: los documentos usan el patron PREFIJO-YYYYMMDD-##### (OC, RM, FC, PED, FV, EN, DV) generado por la BD; la unicidad la respalda una restriccion UNIQUE. (Deuda conocida: no es un consecutivo fiscal formal — ver seccion 7.)

Requisitos no funcionales generales:

- **RNF-GEN-01:** Acceso a datos con consultas parametrizadas (JdbcTemplate); sin concatenacion de entrada de usuario en SQL.
- **RNF-GEN-02:** CORS restringido al origen del frontend (<http://localhost:4200>) con credenciales.
- **RNF-GEN-03:** Defensa en profundidad: la autorizacion se valida en la capa web (RBAC) y de nuevo en la BD (privilegios de grupo + RLS + horario); un fallo de una capa no basta para saltarse la otra.
- **RNF-GEN-04:** Operaciones multi-paso (recepcion, pedido, factura, despacho, devolucion) son transaccionales (@Transactional): o se completan integras o se revierten.

4. Especificaciones por modulo

Cada modulo se describe con la plantilla de 14 apartados: (1) Nombre, (2) Objetivo, (3) Actores, (4) Contexto, (5) Requisitos funcionales, (6) Requisitos no funcionales, (7) Reglas de negocio, (8) Entradas, (9) Salidas, (10) Escenarios (Gherkin), (11) Criterios de aceptacion, (12) Restricciones, (13) Dependencias y (14) Fuera de alcance. El contenido deriva de los endpoints, validaciones y esquema reales.

4.1 Seguridad y Control de Acceso

1. Objetivo

Autenticar a los usuarios (login por email con JWT), autorizar sus acciones mediante RBAC en la capa web y roles de grupo en la base de datos, y restringir la operacion por horario. Incluye la administracion de usuarios y de las ventanas horarias por rol.

2. Actores

- Administrador (unico que crea/elimina usuarios y edita horarios).
- Todos los roles (se autentican y obtienen su JWT).
- Sistema (aspecto PgSessionRoleAspect que asume el rol de motor por transaccion).

3. Contexto

El control de acceso es defensa en profundidad. SecurityConfig declara las reglas RBAC por ruta; JwtAuthenticationFilter valida el token y puebla el SecurityContext; PgSessionRoleAspect ejecuta SET LOCAL ROLE grp_* (y SET LOCAL app.cliente_id para el cliente) al inicio de cada transaccion. La BD aplica ademas privilegios de grupo, RLS y triggers de horario.

4. Requisitos funcionales

- **RF-SEG-01:** El sistema debe autenticar por email y contrasena en POST /api/auth/login y devolver un JWT de acceso y un refresh token (LoginResponseDTO).
- **RF-SEG-02:** El sistema debe renovar la sesion en POST /api/auth/refresh a partir de un refresh token valido.
- **RF-SEG-03:** El sistema debe exponer el usuario autenticado en GET /api/auth/me (username y rol) y cerrar sesion en POST /api/auth/logout.
- **RF-SEG-04:** El Administrador debe poder crear usuarios (POST /api/admin/usuarios y POST /api/auth/register), listarlos (GET), eliminarlos (DELETE /{username|email}) y activar/desactivar (PUT /{username}/toggle-activo).
- **RF-SEG-05:** Al crear un usuario, el rol debe validarse contra la lista ADMIN/GERENTE/VENDEDOR/COMPRAS/BODEGA/DESPACHO/CLIENTE/ANALISTA y el email no debe estar duplicado.

- **RF-SEG-06:** El Administrador debe poder listar, crear y editar ventanas horarias por rol de grupo (GET/POST/PUT /api/admin/horarios) sobre la tabla grupo_horario.
- **RF-SEG-07:** El sistema debe asignar a cada transaccion el rol de motor correspondiente al rol del JWT (SET LOCAL ROLE) y, para el cliente, fijar app.cliente_id para la RLS.

5. Requisitos no funcionales

- **RNF-SEG-01:** Las contraseñas se almacenan con BCrypt; nunca se devuelven en las respuestas.
- **RNF-SEG-02:** El nombre del rol de motor solo proviene del enum DbGroupRole (lista blanca); no se interpola texto del usuario en el SET LOCAL ROLE.
- **RNF-SEG-03:** La sesion es STATELESS: no se guarda estado de sesion en el servidor.
- **RNF-SEG-04:** El SET LOCAL vive solo dentro de la transaccion; la conexion regresa al pool como retailmind_app sin privilegios de negocio.

6. Reglas de negocio

- **RN-SEG-01:** No se puede eliminar al usuario administrador base (DataInitializer.ADMIN_EMAIL).
- **RN-SEG-02:** Solo ADMIN accede a /api/admin/**, /api/auth/register, /api/auth/usuarios, /api/gestion/**, /api/etl/**, /api/init/** y /api/reportes/**.
- **RN-SEG-03:** En grupo_horario, dia_semana debe estar entre 0 (domingo) y 6 (sabado) y el rol_grupo debe ser uno de los grp_* validos; hora_inicio/hora_fin y activo son obligatorios.
- **RN-SEG-04:** El administrador (grp_administrador) y los superusuarios estan exentos de la verificacion horaria; el cliente tiene ventana 24/7.
- **RN-SEG-05:** Sin usuario autenticado no se asume ningun rol de grupo: la transaccion queda limitada a los privilegios directos de retailmind_app (usuario/rol/usuario_rol).

7. Entradas

- Credenciales: email y contraseña (LoginRequestDTO).
- Refresh token (RefreshTokenRequestDTO).
- Alta de usuario: username/email, password, email, rol (y opcional nombre/apellido).
- Ventana horaria: rolGrupo, diaSemana, horaInicio, horaFin, activo.

8. Salidas (incluye mensajes de error)

- JWT de acceso + refresh token e informacion basica del usuario.
- Listado de usuarios (id, username/email, nombre, rol, activo, clientId).
- Mensajes de error: 'Credenciales incorrectas' (401), 'Refresh token invalido o expirado' (401), 'El usuario ya existe', 'Rol invalido: ...', 'No se puede eliminar al usuario admin', 'rol_grupo invalido', 'dia_semana debe estar entre 0 y 6'.
- En BD, fuera de horario: 'Acceso denegado: fuera del horario permitido para el rol ...' (42501).

9. Escenarios (Gherkin)

Escenario 1 - Login exitoso (CU-O-01)

Dado un usuario activo con email y contraseña validos
Cuando envia POST /api/auth/login
Entonces recibe 200 con token de acceso y refresh token
Y el token contiene su rol (p. ej. VENDEDOR).

Escenario 2 - Login fallido

Dado un email inexistente o contraseña incorrecta
Cuando envia POST /api/auth/login
Entonces recibe 401 con mensaje 'Credenciales incorrectas'.

Escenario 3 - Alta de usuario por ADMIN (CU-O-04)

Dado un ADMIN autenticado
Cuando envia POST /api/admin/usuarios con rol 'BODEGA' y un email nuevo
Entonces recibe 200 y el usuario queda creado con contraseña BCrypt.

Escenario 4 - Operacion fuera de horario

Dado un VENDEDOR cuyo horario es L-V 08:00-18:00
Cuando intenta registrar un pedido un domingo
Entonces la BD rechaza la escritura con SQLSTATE 42501 'fuera del horario permitido'.

Escenario 5 - Configurar ventana horaria (CU-O-05)

Dado un ADMIN
Cuando envia POST /api/admin/horarios con rolGrupo=grp_bodega, diaSemana=1, 07:00-17:00
Entonces recibe 201 y la ventana queda registrada en grupo_horario.

10. Criterios de aceptacion

- **CA-SEG-01:** Un usuario con credenciales validas obtiene JWT + refresh y puede consumir endpoints acordes a su rol.
- **CA-SEG-02:** Un rol sin autorizacion para una ruta recibe 401/403 y, si evade la capa web, la BD lo detiene por privilegios/RLS/horario.
- **CA-SEG-03:** El ADMIN puede administrar usuarios y horarios; no puede eliminar al admin base.
- **CA-SEG-04:** Toda escritura fuera de la ventana horaria del rol es rechazada por la BD.

11. Restricciones

- El identificador de login es el email (el campo 'username' se conserva por compatibilidad del frontend).
- El mapeo rol de app -> grp_* es fijo (enum), no configurable en tiempo de ejecucion.
- La validacion horaria depende de la zona horaria del servidor de BD (localtime).

12. Dependencias

- PostgreSQL: tablas usuario, rol, usuario_rol, grupo_horario; funciones esta_en_horario, fn_grupo_actual, fn_cliente_actual, fn_usuario_actual; roles grp_*.
- Spring Security + jjwt; BCryptPasswordEncoder; PgSessionRoleAspect; PostgresUserRepository.

13. Fuera de alcance

- Autoservicio de registro publico (el alta la hace el ADMIN).
- Recuperacion de contrasena / verificacion de email (existen tablas token_recuperacion y campos email_verificado, pero el flujo no esta expuesto — por confirmar).
- Bloqueo automatico por intentos fallidos: los campos intentos_fallidos/bloqueado_hasta existen, pero su aplicacion efectiva queda por confirmar.

4.2 Catalogo

1. Objetivo

Mantener el catalogo comercial (categorias, marcas, productos, variantes y atributos) y exponer su consulta publica. El catalogo alimenta las ventas y es el unico modulo con lectura sin autenticacion.

2. Actores

- Administrador (gestiona todo el catalogo bajo /api/admin/catalogo).
- Publico / visitante (consulta el catalogo sin login).
- Cliente autenticado (consulta y genera eventos de comportamiento).

3. Contexto

La consulta publica vive en /api/catalogo/** (permitida sin auth en SecurityConfig). La gestion vive en /api/admin/catalogo/** (solo ADMIN + grp_administrador en BD). Los productos tienen variantes (SKU, precio, costo) y pueden asociarse a categorias, marcas y valores de atributo.

4. Requisitos funcionales

- **RF-CAT-01:** El sistema debe listar productos con filtros por categoria, marca y rango de precio, paginados (GET /api/catalogo/productos?categoria_id&brand&min_price&max_price&page&size).
- **RF-CAT-02:** El sistema debe devolver el detalle de un producto (GET /api/catalogo/productos/{id}) y 404 si no existe.
- **RF-CAT-03:** El sistema debe listar categorias y marcas publicas (GET /api/catalogo/categorias, /api/catalogo/marcas).
- **RF-CAT-04:** El sistema debe registrar eventos de comportamiento (POST /api/catalogo/eventos: user_id, product_id, user_action, channel, price, session_id).
- **RF-CAT-05:** El Administrador debe poder crear, editar y activar/desactivar categorias (/api/admin/catalogo/categorias) y marcas (/marcas).
- **RF-CAT-06:** El Administrador debe poder crear, editar, ver y activar/desactivar productos (/api/admin/catalogo/productos), incluida su asociacion a categorias.
- **RF-CAT-07:** El Administrador debe poder crear y editar variantes con SKU, precio y costo, y activarlas/desactivarlas (/api/admin/catalogo/variantes).
- **RF-CAT-08:** El Administrador debe poder definir atributos y sus valores y asociarlos a variantes (/api/admin/catalogo/atributos, /variantes/{id}/atributos).

5. Requisitos no funcionales

- **RNF-CAT-01:** La consulta publica no requiere autenticacion pero es de solo lectura.
- **RNF-CAT-02:** El listado es paginado (page/size, size por defecto 20) para acotar la carga.
- **RNF-CAT-03:** Las escrituras de catalogo corren bajo grp_administrador y respetan horario/RLS.

6. Reglas de negocio

- **RN-CAT-01:** El SKU de variante y los slugs de producto/categoria/marca son unicos (restriccion de esquema); una variante puede marcarse como predeterminada.
- **RN-CAT-02:** Solo se muestran en tienda los productos publicados/activos; el filtro de precio opera sobre el precio de la variante.
- **RN-CAT-03:** Activar/desactivar es un borrado logico (campo activo/publicado); no se elimina fisicamente el registro.
- **RN-CAT-04:** El registro de eventos no bloquea la navegacion: es de apoyo analitico.

7. Entradas

- Filtros de busqueda: categoria_id, brand, min_price, max_price, page, size.
- Alta/edicion de categoria (nombre, slug, descripcion, padreId), marca (nombre, slug, descripcion).
- Alta/edicion de producto (nombre, slug, marcaId, descripciones, publicado, categorialds).
- Alta/edicion de variante (sku, precio, costo, codigoBarras, esPredeterminada).
- Atributos (codigo, nombre, tipo) y valores; asociacion valorAtributId a variante.

8. Salidas (incluye mensajes de error)

- Pagina de productos y detalle; listas de categorias y marcas.
- IDs de los recursos creados (201 Created).
- Mensajes de error: 404 en detalle inexistente; errores de validacion de esquema (slug/SKU duplicado).

9. Escenarios (Gherkin)

Escenario 1 - Consultar catalogo publico (CU-O-06)

Dado un visitante sin sesion

Cuando pide GET /api/catalogo/productos?categoria_id=3&max_price=100&page=0&size=20

Entonces recibe 200 con la pagina de productos que cumplen el filtro.

Escenario 2 - Detalle inexistente

Dado un identificador de producto que no existe

Cuando pide GET /api/catalogo/productos/{id}

Entonces recibe 404 Not Found.

Escenario 3 - Crear producto (CU-O-09)

Dado un ADMIN autenticado

Cuando envia POST /api/admin/catalogo/productos con nombre, slug y categorialds

Entonces recibe 201 con el id del producto creado.

Escenario 4 - Crear variante con SKU y precio (CU-0-10)

Dado un producto existente

Cuando envia POST /api/admin/catalogo/productos/{id}/variantes con sku y precio

Entonces recibe 201 y la variante queda disponible para venta.

Escenario 5 - Registrar evento de vista

Dado un usuario navegando un producto

Cuando envia POST /api/catalogo/eventos con user_action='view'

Entonces recibe 200 success y el evento queda registrado.

10. Criterios de aceptacion

- **CA-CAT-01:** El publico puede consultar productos, categorias y marcas sin autenticarse.
- **CA-CAT-02:** El ADMIN puede dar de alta un producto con variantes y publicarlo.
- **CA-CAT-03:** Un detalle inexistente responde 404 y no expone error interno.
- **CA-CAT-04:** Desactivar una entidad la retira de la tienda sin borrarla.

11. Restricciones

- La gestion de catalogo es exclusiva de ADMIN; el resto de roles solo consulta.
- El precio de venta reside en la variante (producto_variante.precio), no en el producto.

12. Dependencias

- PostgreSQL: producto, producto_variante, categoria, marca, atributo, valor_atributo, variante_valor_atributo, producto_categoria, producto_imagen.
- ClickHouse/registro de eventos para analitica (no bloqueante).

13. Fuera de alcance

- Gestion de imagenes/multimedia enriquecida y SEO avanzado.
- Precios por lista/segmento, promociones y cupones (existen tablas promocion/cupon pero no en este alcance operativo).

4.3 Proveedores y Compras

1. Objetivo

Ejecutar el ciclo Procure-to-Pay: emitir ordenes de compra a proveedores, recibir la mercancia (actualizando stock y kardex) y liquidar la deuda mediante pagos contra las cuentas por pagar. La facturación de compra se detalla en el módulo 4.6.

2. Actores

- Encargado de Compras (emite ordenes, factura).
- Encargado de Bodega (registra recepciones).
- Gerente (aprobación/lectura).
- Administrador (acceso total).

3. Contexto

Bajo `/api/compras/**` (roles ADMIN/GERENTE/COMPRAS/BODEGA; la BD afina por SET LOCAL ROLE). La orden nace 'enviada'; la recepción mueve stock por StockService y avanza la orden a 'recibida' o 'recibida_parcial'. Los subtotales de detalle son columnas generadas y los totales de cabecera los ponen triggers.

4. Requisitos funcionales

- **RF-CMP-01:** El sistema debe emitir una orden de compra (POST `/api/compras/ordenes`) con proveedor, bodega, moneda opcional, fecha de entrega esperada e items (variante, cantidad, precio unitario, %IVA).
- **RF-CMP-02:** El sistema debe generar el número de orden (patrón OC-YYYYMMDD-#####) y calcular el impuesto por ítem; el subtotal y los totales los resuelven columnas generadas y triggers.
- **RF-CMP-03:** El sistema debe listar ordenes (GET `/api/compras/ordenes`) y devolver el detalle de una orden con sus líneas y cantidades recibidas (GET `/api/compras/ordenes/{id}`).
- **RF-CMP-04:** El sistema debe registrar la recepción de mercancía (POST `/api/compras/ordenes/{id}/recepciones`): por cada línea suma stock en la bodega de la orden, registra el kardex (entrada_compra) y actualiza cantidad_recibida.
- **RF-CMP-05:** El sistema debe listar cuentas por pagar (GET `/api/compras/cuentas-por-pagar`) y registrar pagos a proveedor (POST `/api/compras/cuentas-por-pagar/{id}/pagos`).
- **RF-CMP-06:** Al pagar, el sistema debe descontar del saldo pendiente y marcar la cuenta como 'parcial' o 'pagada', y la factura como 'pagada_parcial' o 'pagada'.

5. Requisitos no funcionales

- **RNF-CMP-01:** La recepción es transaccional: stock, kardex y estado de orden se actualizan de forma atómica.

- **RNF-CMP-02:** La fila de inventario se bloquea con SELECT ... FOR UPDATE antes de moverla, evitando condiciones de carrera.
- **RNF-CMP-03:** Idempotencia: una orden no puede facturarse dos veces (guardia explicita).

6. Reglas de negocio

- **RN-CMP-01:** La orden requiere al menos un ítem; cada ítem exige cantidad > 0 y precio unitario > 0.
- **RN-CMP-02:** No se puede recibir sobre una orden 'recibida' (completa) ni 'cancelada'.
- **RN-CMP-03:** La cantidad recibida por línea debe ser > 0 y no puede superar lo pendiente (cantidad - cantidad_recibida).
- **RN-CMP-04:** La orden pasa a 'recibida' cuando todas las líneas están completas; si no, a 'recibida_parcial'.
- **RN-CMP-05:** El pago debe ser > 0 y no puede exceder el saldo pendiente; una cuenta liquidada no admite más pagos.
- **RN-CMP-06:** El vencimiento de la factura/cuenta se calcula con proveedor.dias_credito desde la fecha de emisión.

7. Entradas

- Orden: proveedorId, bodegaId, monedaId?, fechaEntregaEsperada, observacion, items[variantId, cantidad, precioUnitario, ivaPorcentaje?].
- Recepcion: items[ordenCompraDetalleId, cantidadRecibida, cantidadRechazada?, motivoRechazo?].
- Pago: monto, metodoPagoid, referencia.

8. Salidas (incluye mensajes de error)

- Orden creada con número, estado y totales (leídos tras el trigger).
- Recepcion con número (RM-...) y estado resultante de la orden.
- Saldo pendiente y estado de la cuenta tras el pago.
- Mensajes de error: 'La orden requiere al menos un ítem', 'La cantidad ... debe ser mayor a cero', 'solo quedan N pendientes por recibir', 'ya fue recibida completamente', 'El pago excede el saldo pendiente'.

9. Escenarios (Gherkin)

Escenario 1 - Emitir orden de compra (CU-O-11)

Dado un COMPRAS autenticado en horario

Cuando envía POST /api/compras/ordenes con proveedor, bodega e ítems válidos

Entonces recibe 201 con la orden en estado 'enviada' y sus totales calculados.

Escenario 2 - Recepcion parcial (CU-O-13)

Dada una orden 'enviada' con 10 unidades de un SKU

Cuando se reciben 4 unidades

Entonces el stock sube 4, se registra el kardex 'entrada_compra' y la orden queda 'recibida_parcial'.

Escenario 3 - Recibir mas de lo pendiente

Dada una linea con 6 unidades pendientes

Cuando se intenta recibir 8

Entonces se rechaza con 'solo quedan 6 pendientes por recibir'.

Escenario 4 - Pago que liquida la cuenta (CU-O-14)

Dada una cuenta por pagar con saldo 100

Cuando se registra un pago de 100

Entonces la cuenta queda 'pagada' y la factura 'pagada'.

Escenario 5 - Pago que excede el saldo

Dada una cuenta con saldo 50

Cuando se intenta pagar 80

Entonces se rechaza con 'El pago (80) excede el saldo pendiente (50)'.

10. Criterios de aceptacion

- **CA-CMP-01:** Una orden valida se crea con numero unico y totales resueltos por la BD.
- **CA-CMP-02:** La recepcion incrementa stock y deja rastro en el kardex con referencia a la recepcion.
- **CA-CMP-03:** El sistema impide recibir de mas y facturar dos veces la misma orden.
- **CA-CMP-04:** Los pagos ajustan saldo y estados de cuenta y factura de forma consistente.

11. Restricciones

- La app no calcula totales de cabecera ni subtotales: los resuelve la BD.
- La recepcion afecta la bodega declarada en la orden, no una arbitraria.

12. Dependencias

- PostgreSQL: orden_compra(_detalle), recepcion_mercancia(_detalle), factura_compra(_detalle), cuenta_por_pagar, pago_proveedor, proveedor, moneda, tipo_movimiento, inventario, movimiento_inventario.
- StockService (movimiento de stock + kardex).

13. Fuera de alcance

- Aprobacion formal multinivel de ordenes (flujo de aprobacion) — el estado 'enviada' no modela una aprobacion explicita.
- Devolucion a proveedor (existe tipo salida_devolucion_proveedor pero no hay endpoint).

4.4 Inventario y Bodegas

1. Objetivo

Controlar las existencias por variante y bodega y registrar sus movimientos (kardex). El caso operativo expuesto es la transferencia de stock entre bodegas; la consulta de existencias apoya a los demas modulos.

2. Actores

- Encargado de Bodega (ejecuta transferencias).
- Gerente / Administrador (lectura y control).
- Sistema (StockService, invocado por compras, ventas y devoluciones).

3. Contexto

Bajo `/api/inventario/**` (roles ADMIN/GERENTE/BODEGA). Toda variacion de stock pasa por StockService: upsert de inventario, bloqueo FOR UPDATE, fila en movimiento_inventario y update del stock. transferencia_bodega es solo cabecera: la variante y cantidad quedan en los dos movimientos del kardex y en la observacion.

4. Requisitos funcionales

- **RF-INV-01:** El sistema debe transferir stock entre dos bodegas (POST `/api/inventario/transferencias`) generando una salida en origen (`salida_transferencia`) y una entrada en destino (`entrada_transferencia`).
- **RF-INV-02:** El sistema debe listar las transferencias realizadas (GET `/api/inventario/transferencias`) con bodega origen/destino y solicitante.
- **RF-INV-03:** El sistema debe permitir consultar existencias por variante y/o bodega (GET `/api/referencias/stock`).
- **RF-INV-04:** Cada movimiento debe registrar `stock_anterior`, `stock_nuevo`, cantidad, `tipo_movimiento`, usuario y referencia polimorfica en `movimiento_inventario` (kardex).

5. Requisitos no funcionales

- **RNF-INV-01:** La transferencia es transaccional: si falla la entrada en destino, se revierte la salida en origen.
- **RNF-INV-02:** StockService exige transaccion activa (Propagation.MANDATORY): siempre corre bajo el SET LOCAL ROLE del caso de uso que lo invoca.
- **RNF-INV-03:** El signo del movimiento se toma del catalogo `tipo_movimiento` (lista blanca), no de texto libre.

6. Reglas de negocio

- **RN-INV-01:** La bodega origen y la bodega destino deben ser distintas.

- **RN-INV-02:** La cantidad a mover debe ser > 0 .
- **RN-INV-03:** No se permite dejar stock negativo: si el movimiento resultara en $\text{stock} < 0$ se rechaza con 'Stock insuficiente para SKU ... en bodega ...'.
- **RN-INV-04:** El tipo de movimiento debe existir y estar activo en tipo_movimiento; el factor (+1/-1) determina si suma o resta.

7. Entradas

- Transferencia: varianteld, bodegaOrigenId, bodegaDestinoId, cantidad, observacion.
- Consulta de stock: varianteld? y/o bodegald? (filtros opcionales).

8. Salidas (incluye mensajes de error)

- Transferencia con id, SKU, cantidad, stock resultante en origen y destino, estado 'recibida'.
- Listado de existencias (variante, bodega, stock_actual).
- Mensajes de error: 'La bodega origen y destino deben ser distintas', 'La cantidad debe ser mayor a cero', 'Stock insuficiente para SKU ...'.

9. Escenarios (Gherkin)

Escenario 1 - Transferir stock (CU-O-15)

Dado un SKU con 20 unidades en la bodega A

Cuando se transfieren 5 a la bodega B

Entonces A queda con 15, B con 5 y el kardex registra salida y entrada por transferencia.

Escenario 2 - Transferir a la misma bodega

Dada bodega origen = bodega destino

Cuando se intenta transferir

Entonces se rechaza con 'La bodega origen y destino deben ser distintas'.

Escenario 3 - Stock insuficiente

Dado un SKU con 3 unidades en origen

Cuando se intentan transferir 5

Entonces se rechaza con 'Stock insuficiente para SKU ...'.

Escenario 4 - Consultar existencias (CU-O-17)

Dado un usuario autenticado

Cuando pide GET /api/referencias/stock?varianteld=10

Entonces recibe el stock de esa variante por bodega.

10. Criterios de aceptacion

- **CA-INV-01:** Una transferencia valida ajusta ambos stocks y deja dos movimientos en el kardex.
- **CA-INV-02:** El sistema nunca permite stock negativo.
- **CA-INV-03:** Las existencias consultadas coinciden con la suma de movimientos del kardex.

11. Restricciones

- transferencia_bodega no tiene tabla de detalle: una transferencia registra una sola variante por cabecera (la cantidad/variante viven en el kardex y la observacion).
- No hay reserva de stock en el flujo de venta (se descuenta directo).

12. Dependencias

- PostgreSQL: inventario, movimiento_inventario, transferencia_bodega, bodega, producto_variante, tipo_movimiento.
- StockService.

13. Fuera de alcance

- Ajustes de inventario por conteo fisico (tabla ajuste_inventario y tipos entrada_ajuste/salida_ajuste existen, pero sin endpoint).
- Gestion de lotes y ubicaciones detalladas (lote, ubicacion_bodega existen; no expuestas).
- Endpoint de consulta del kardex (movimiento_inventario se escribe pero no se lee via API).

4.5 Ventas y Pedidos

1. Objetivo

Iniciar el ciclo Order-to-Cash: registrar pedidos con su detalle, descontar stock al confirmarlos y mantener el historial de estados. La facturación, el despacho y la devolución se especifican en los módulos 4.6, 4.7 y 4.8.

2. Actores

- Vendedor (registra pedidos).
- Cliente (consulta sus propios pedidos; RLS lo aísla).
- Gerente / Administrador (consulta global).

3. Contexto

Bajo `/api/ventas/**` (ADMIN/GERENTE/VENDEDOR/DESPACHO/CLIENTE) y `/api/pedidos/**`. El pedido nace 'confirmado' y DESCUENTA stock directo (`salida_venta`) por `StockService` (decisión documentada: mas simple que reservar). Cada cambio de estado deja fila en `historial_estado_pedido`. El cliente solo ve sus filas por RLS (`app.cliente_id`).

4. Requisitos funcionales

- **RF-VEN-01:** El sistema debe crear un pedido (POST `/api/ventas/pedidos`) con cliente, bodega, canal e items (variante, cantidad); nace en estado 'confirmado'.
- **RF-VEN-02:** Por cada item, el sistema debe tomar un snapshot de nombre, SKU y precio vigente de la variante activa y descontar stock (`salida_venta`) con kardex.
- **RF-VEN-03:** El sistema debe registrar el historial de estados del pedido (`historial_estado_pedido`) en cada transición.
- **RF-VEN-04:** El sistema debe listar pedidos operativos (GET `/api/ventas/pedidos`) y su detalle con líneas e historial (GET `/api/ventas/pedidos/{id}`).
- **RF-VEN-05:** El Administrador debe poder consultar todos los pedidos (GET `/api/pedidos/admin/todos`).
- **RF-VEN-06:** El Cliente debe poder consultar sus pedidos (GET `/api/pedidos/{userId}`).

5. Requisitos no funcionales

- **RNF-VEN-01:** La creación del pedido es transaccional: detalle y descuento de stock se confirman juntos.
- **RNF-VEN-02:** El canal se normaliza a un valor válido (web/tienda/telefono; por defecto web).
- **RNF-VEN-03:** El aislamiento del cliente lo garantiza la RLS de la BD, no solo la capa web.

6. Reglas de negocio

- **RN-VEN-01:** El pedido requiere al menos un item; cada cantidad debe ser > 0 .

- **RN-VEN-02:** La variante debe existir y estar activa; si no, se rechaza con 'no existe o esta inactivo'.
- **RN-VEN-03:** El descuento de stock no puede dejar existencias negativas (via StockService).
- **RN-VEN-04:** El subtotal de linea es columna generada y el total del pedido lo calcula un trigger; la app solo los lee.
- **RN-VEN-05:** El impuesto por linea se calcula al 15% (IVA por defecto) sobre el importe.

7. Entradas

- Pedido: clienteld, bodegald, canal, items[varianteld, cantidad].
- Consulta: id de pedido, userld de cliente.

8. Salidas (incluye mensajes de error)

- Pedido con numero (PED-...), estado, totales, detalle e historial.
- Listas de pedidos (numero, estado, total, cliente).
- Mensajes de error: 'El pedido requiere al menos un ítem', 'La cantidad ... debe ser mayor a cero', 'El producto (variante N) no existe o esta inactivo', 'Stock insuficiente ...'.

9. Escenarios (Gherkin)

Escenario 1 - Realizar pedido (CU-O-18)

Dado un VENDEDOR en horario y una variante activa con stock
 Cuando envia POST /api/ventas/pedidos con el ítem
 Entonces recibe 201, el pedido queda 'confirmado' y el stock se descuenta.

Escenario 2 - Producto inactivo

Dada una variante inactiva
 Cuando se intenta pedir
 Entonces se rechaza con 'no existe o esta inactivo'.

Escenario 3 - Consultar mis pedidos como cliente (CU-O-20)

Dado un CLIENTE autenticado
 Cuando pide GET /api/pedidos/{suUserId}
 Entonces recibe unicamente sus pedidos (RLS por app.cliente_id).

Escenario 4 - Consulta global de pedidos (CU-O-19)

Dado un ADMIN
 Cuando pide GET /api/pedidos/admin/todos
 Entonces recibe todos los pedidos del sistema.

10. Criterios de aceptacion

- **CA-VEN-01:** Un pedido valido se crea 'confirmado', con snapshot de precio y stock descontado.
- **CA-VEN-02:** El cliente solo ve sus pedidos; el ADMIN ve todos.
- **CA-VEN-03:** El historial de estados refleja cada transicion del pedido.

11. Restricciones

- El pedido descuenta stock directo al confirmar (no reserva).
- El IVA por defecto es 15% (parametro del codigo).

12. Dependencias

- PostgreSQL: pedido(_detalle), estado_pedido, historial_estado_pedido, cliente, producto_variante, moneda, inventario, movimiento_inventario.
- StockService; PedidosService/VentasService.

13. Fuera de alcance

- Carrito/checkout de autoservicio del cliente como flujo operativo nucleo (existe carrito pero no es el canal transaccional documentado aqui).
- Cupones, promociones y costos de envio calculados dinamicamente.

4.6 Facturacion (compra y venta)

1. Objetivo

Emitir los comprobantes de las dos puntas del negocio: la factura de compra (que abre la cuenta por pagar al proveedor) y la factura de venta al cliente, ambas con su representacion imprimible en PDF.

2. Actores

- Encargado de Compras (factura de compra).
- Vendedor (factura de venta).
- Gerente / Administrador.

3. Contexto

La factura de compra se genera desde una orden (POST /api/compras/ordenes/{id}/facturas), copia el detalle pactado y abre una cuenta_por_pagar. La factura de venta se genera desde un pedido (POST /api/ventas/pedidos/{id}/factura) con snapshot de datos fiscales del cliente. Ambos numeros los genera el sistema y ambos totales los ponen triggers.

4. Requisitos funcionales

- **RF-FAC-01:** El sistema debe registrar la factura de compra de una orden, copiando su detalle y generando numero (FC-...); el subtotal es generado y el total lo pone el trigger.
- **RF-FAC-02:** Al facturar la compra, el sistema debe abrir una cuenta_por_pagar con monto_original = total y vencimiento = fecha + dias_credito del proveedor.
- **RF-FAC-03:** El sistema debe emitir la factura de venta de un pedido (numero FV-...), copiando el detalle del pedido como snapshot y guardando razon_social, identificacion y direccion_facturacion.
- **RF-FAC-04:** El sistema debe exponer el detalle de cada factura (GET /api/compras/facturas/{id}, GET /api/ventas/facturas/{id}).
- **RF-FAC-05:** El sistema debe generar el PDF imprimible de la factura de compra y de venta (GET .../facturas/{id}/pdf, Content-Type application/pdf).

5. Requisitos no funcionales

- **RNF-FAC-01:** Idempotencia: una orden/pedido no puede facturarse dos veces (guardia explicita).
- **RNF-FAC-02:** El PDF se entrega inline con nombre de archivo derivado del id de factura.
- **RNF-FAC-03:** Los importes mostrados provienen de la BD (totales por trigger), no de calculo en cliente.

6. Reglas de negocio

- **RN-FAC-01:** No se puede facturar un pedido en estado 'cancelado' o 'devuelto'.
- **RN-FAC-02:** Si la orden ya tiene factura, se rechaza con 'ya tiene la factura ... registrada'.

- **RN-FAC-03:** Si el pedido ya fue facturado, se rechaza con 'ya fue facturado (factura ...)'.
- **RN-FAC-04:** Al emitir la factura de venta el pedido pasa a estado 'pagado' (historial).
- **RN-FAC-05:** La identificación del cliente usa numero_identificacion o '9999999999' (consumidor final) cuando no exista.

7. Entradas

- Factura de compra: id de la orden de compra.
- Factura de venta: id del pedido.
- PDF: id de la factura (compra o venta).

8. Salidas (incluye mensajes de error)

- Factura (numero, estado, fechas, subtotal, impuesto, total, detalle).
- Cuenta por pagar asociada (saldo, estado) en la factura de compra.
- Documento PDF (application/pdf).
- Mensajes de error: 'No se puede facturar un pedido en estado ...', 'ya tiene la factura ... registrada', 'ya fue facturado'.

9. Escenarios (Gherkin)

Escenario 1 - Registrar factura de compra (CU-O-21)

Dada una orden de compra recibida

Cuando se envia POST /api/compras/ordenes/{id}/facturas

Entonces se crea la factura FC-..., se copia el detalle y se abre la cuenta por pagar.

Escenario 2 - Emitir factura de venta (CU-O-22)

Dado un pedido facturable

Cuando se envia POST /api/ventas/pedidos/{id}/factura

Entonces se crea la factura FV-... con snapshot fiscal y el pedido pasa a 'pagado'.

Escenario 3 - Doble facturación

Dado un pedido ya facturado

Cuando se intenta facturar de nuevo

Entonces se rechaza con 'ya fue facturado (factura ...)'.

Escenario 4 - Generar PDF (CU-O-23)

Dada una factura existente

Cuando se pide GET /api/ventas/facturas/{id}/pdf

Entonces se descarga un PDF inline con el comprobante.

Escenario 5 - Facturar pedido devuelto

Dado un pedido en estado 'devuelto'

Cuando se intenta facturar

Entonces se rechaza con 'No se puede facturar un pedido en estado devuelto'.

10. Criterios de aceptacion

- **CA-FAC-01:** Facturar una compra abre la cuenta por pagar con el vencimiento correcto.
- **CA-FAC-02:** Facturar una venta congela los datos fiscales y avanza el pedido a 'pagado'.
- **CA-FAC-03:** Ninguna orden/pedido se factura dos veces.
- **CA-FAC-04:** El PDF de ambas facturas se genera y descarga correctamente.

11. Restricciones

- El numero de factura sigue el patron PREFIJO-YYYYMMDD-##### (no es serie fiscal secuencial).
- El campo clave_acceso de factura_venta existe pero no se emite (sin facturacion electronica).

12. Dependencias

- PostgreSQL: factura_compra(_detalle), factura_venta(_detalle), cuenta_por_pagar, pedido, orden_compra, cliente, proveedor, moneda.
- iText (FacturaCompraPdfService, FacturaVentaPdfService, DocumentoPdfService).

13. Fuera de alcance

- Facturacion electronica / firma / clave de acceso (SRI).
- Notas de credito formales (la devolucion no emite comprobante fiscal).

4.7 Logística y Envíos

1. Objetivo

Despachar los pedidos generando el envío con su guía, el detalle de lo enviado y el primer evento de seguimiento, y permitir consultar el rastreo del envío.

2. Actores

- Encargado de Despacho (despacha y da seguimiento).
- Vendedor / Gerente / Administrador (según RBAC).
- Cliente (consulta el estado de su envío).

3. Contexto

Bajo `/api/ventas/**`. El despacho crea una fila en envío (número EN-..., guía GUIA-...), copia el detalle del pedido a `envio_detalle`, registra el primer `seguimiento_envio` y avanza el pedido a 'despachado'. La dirección de entrega se resuelve de la dirección predeterminada del cliente o 'Retiro en tienda'.

4. Requisitos funcionales

- **RF-LOG-01:** El sistema debe despachar un pedido (POST `/api/ventas/pedidos/{id}/despacho`) con transportista, método de envío y bodega, generando envío con número y guía.
- **RF-LOG-02:** El sistema debe copiar el detalle del pedido a `envio_detalle` y registrar el primer evento en `seguimiento_envio` (estado 'en_transito').
- **RF-LOG-03:** El despacho debe avanzar el pedido a estado 'despachado' con su registro en el historial.
- **RF-LOG-04:** El sistema debe exponer el envío (GET `/api/ventas/envios/{id}`) y su historial de seguimiento (GET `/api/ventas/envios/{id}/seguimiento`).

5. Requisitos no funcionales

- **RNF-LOG-01:** El despacho es transaccional: envío, detalle, seguimiento y estado del pedido se confirman juntos.
- **RNF-LOG-02:** La guía se deriva del número de envío para trazabilidad.

6. Reglas de negocio

- **RN-LOG-01:** Solo se puede despachar un pedido en estado 'confirmado', 'pagado' o 'en_preparacion'.
- **RN-LOG-02:** No se puede despachar un pedido ya 'despachado' o 'entregado' (se informa la guía existente).
- **RN-LOG-03:** Si el cliente no tiene dirección predeterminada, la entrega se marca como 'Retiro en tienda'.

7. Entradas

- Despacho: transportistaId, metodoEnvioId, bodegaId?, observacion.
- Consulta: id del envio.

8. Salidas (incluye mensajes de error)

- Envio con numero, guia, estado, transportista, metodo y detalle enviado.
- Historial de seguimiento (estado, descripcion, ubicacion, fecha).
- Mensajes de error: 'El pedido ya fue despachado (guia ...)', 'No se puede despachar un pedido en estado ...'.

9. Escenarios (Gherkin)

Escenario 1 - Despachar pedido (CU-O-24)

Dado un pedido 'confirmado'

Cuando se envia POST /api/ventas/pedidos/{id}/despacho con transportista y metodo

Entonces se crea el envio con guia, el pedido pasa a 'despachado' y se registra el primer seguimiento.

Escenario 2 - Doble despacho

Dado un pedido ya 'despachado'

Cuando se intenta despachar de nuevo

Entonces se rechaza informando la guia existente.

Escenario 3 - Consultar seguimiento (CU-O-25)

Dado un envio en transito

Cuando se pide GET /api/ventas/envios/{id}/seguimiento

Entonces se devuelve la lista de eventos de rastreo ordenada.

Escenario 4 - Despacho con retiro en tienda

Dado un cliente sin direccion predeterminada

Cuando se despacha su pedido

Entonces la direccion de entrega queda como 'Retiro en tienda'.

10. Criterios de aceptacion

- CA-LOG-01: Un pedido en estado valido se despacha generando envio, guia y seguimiento.
- CA-LOG-02: Un pedido no se despacha dos veces.
- CA-LOG-03: El seguimiento del envio es consultable y ordenado cronologicamente.

11. Restricciones

- El seguimiento inicial se crea con ubicacion fija 'Bodega RetailMind - Quevedo'.

- La actualizacion posterior de estados del envio (entregado, incidencias) no esta expuesta como endpoint propio (por confirmar).

12. Dependencias

- PostgreSQL: envio(_detalle), seguimiento_envio, transportista, metodo_envio, pedido, pedido_detalle, direccion, cliente.

13. Fuera de alcance

- Integracion con couriers externos / tracking en tiempo real.
- Calculo de tarifas de envio por zona (tarifa_envio/zona_envio existen; no se aplican aqui).

4.8 Devoluciones

1. Objetivo

Procesar devoluciones (RMA) de pedidos: registrar los items devueltos con su motivo, estado y accion, reingresar el stock al inventario y dejar el pedido en estado 'devuelto'.

2. Actores

- Vendedor / Encargado de Despacho (gestiona la devolucion).
- Gerente / Administrador.
- Cliente (origen de la solicitud).

3. Contexto

Bajo `/api/ventas/**` . La devolucion valida el motivo contra `motivo_devolucion`, registra devolucion + `devolucion_detalle`, reingresa stock (`entrada_devolucion_cliente`) por `StockService` y cambia el pedido a 'devuelto'. El monto total de la devolucion se calcula sobre el precio de las lineas devueltas.

4. Requisitos funcionales

- **RF-DEV-01:** El sistema debe procesar una devolucion (POST `/api/ventas/pedidos/{id}/devolucion`) con motivo, bodega, descripcion e items (linea de pedido, cantidad, estado, accion).
- **RF-DEV-02:** Por cada item devuelto, el sistema debe reingresar stock (`entrada_devolucion_cliente`) con kardex y acumular el monto de la devolucion.
- **RF-DEV-03:** El sistema debe cambiar el pedido a 'devuelto' y registrar el historial.
- **RF-DEV-04:** El sistema debe exponer el detalle de la devolucion (GET `/api/ventas/devoluciones/{id}`).

5. Requisitos no funcionales

- **RNF-DEV-01:** La devolucion es transaccional: detalle, stock y estado del pedido se confirman juntos.
- **RNF-DEV-02:** El estado del producto y la accion se normalizan a valores validos (nuevo/abierto/danado; reembolso/cambio/credito).

6. Reglas de negocio

- **RN-DEV-01:** La devolucion requiere al menos un item; cada cantidad debe ser > 0 .
- **RN-DEV-02:** No se puede devolver un pedido en estado 'cancelado' o 'pendiente'.
- **RN-DEV-03:** El motivo debe existir y estar activo en `motivo_devolucion` (arrepentimiento, producto_danado, no_corresponde, talla_incorrecta).
- **RN-DEV-04:** No se puede devolver mas de lo comprado sumando TODAS las devoluciones previas de la misma linea.

7. Entradas

- Devolucion: motivoCodigo, bodegalId, descripcion, items[pedidoDetalleId, cantidad, estadoProducto, accion].
- Consulta: id de la devolucion.

8. Salidas (incluye mensajes de error)

- Devolucion con numero (DV-...), estado, monto total, motivo y detalle.
- Mensajes de error: 'La devolucion requiere al menos un item', 'No se puede devolver un pedido en estado ...', 'El motivo ... no existe o esta inactivo', 'se compraron N y ya se devolvieron M'.

9. Escenarios (Gherkin)

Escenario 1 - Procesar devolucion (CU-O-26)

Dado un pedido despachado con 3 unidades de un SKU

Cuando se devuelven 2 con motivo 'talla_incorrecta'

Entonces se registra la devolucion DV-..., el stock sube 2 y el pedido queda 'devuelto'.

Escenario 2 - Motivo invalido

Dado un motivo que no existe

Cuando se intenta procesar la devolucion

Entonces se rechaza con 'El motivo ... no existe o esta inactivo'.

Escenario 3 - Devolver mas de lo comprado

Dada una linea de 3 unidades con 2 ya devueltas

Cuando se intentan devolver 2 mas

Entonces se rechaza con 'se compraron 3 y ya se devolvieron 2'.

Escenario 4 - Consultar devolucion (CU-O-27)

Dada una devolucion registrada

Cuando se pide GET /api/ventas/devoluciones/{id}

Entonces se devuelve su cabecera y detalle.

10. Criterios de aceptacion

- CA-DEV-01: Una devolucion valida reingresa stock y deja el pedido en 'devuelto'.
- CA-DEV-02: El sistema impide devolver mas unidades de las compradas.
- CA-DEV-03: Solo se aceptan motivos activos del catalogo.

11. Restricciones

- La devolucion no emite nota de credito ni comprobante fiscal.

- El reembolso/credito economico no se ejecuta en un modulo de pagos (solo se registra la accion).

12. Dependencias

- PostgreSQL: devolucion(_detalle), motivo_devolucion, pedido(_detalle), inventario, movimiento_inventario, tipo_movimiento.
- StockService; VentasService.

13. Fuera de alcance

- Devolucion a proveedor (salida_devolucion_proveedor sin endpoint).
- Gestion de reembolsos monetarios efectivos (tabla reembolso existe; no en este alcance).

5. Casos de uso del nivel operativo

Se documentan los 27 casos de uso operativos, clasificados por paquete. Cada caso indica su identificador (CU-O-XX), actor principal, descripción breve y flujo principal (endpoint/es y pasos). Todos están respaldados por endpoints reales del backend y pantallas de features/operativo.

5.1 Seguridad y Control de Acceso

CU-O-01 — Iniciar sesión

- **Actor:** Todos los roles
- **Descripción:** Autenticarse por email y contraseña para obtener un JWT.
- **Flujo principal:** POST /api/auth/login -> valida credenciales (BCrypt) -> emite token de acceso + refresh.

CU-O-02 — Renovar sesión

- **Actor:** Todos los roles
- **Descripción:** Obtener un nuevo token de acceso a partir del refresh token.
- **Flujo principal:** POST /api/auth/refresh -> valida refresh -> emite nuevo JWT.

CU-O-03 — Cerrar sesión

- **Actor:** Todos los roles
- **Descripción:** Finalizar la sesión del cliente (descartar token).
- **Flujo principal:** POST /api/auth/logout -> confirma cierre (STATELESS).

CU-O-04 — Gestionar usuarios y roles

- **Actor:** Administrador
- **Descripción:** Crear, listar, activar/desactivar y eliminar usuarios asignando su rol.
- **Flujo principal:** POST/GET/DELETE /api/admin/usuarios, PUT /toggle-activo, POST /api/auth/register -> valida rol y email único -> persiste con BCrypt.

CU-O-05 — Configurar horarios de acceso

- **Actor:** Administrador
- **Descripción:** Definir las ventanas horarias permitidas por rol de grupo.
- **Flujo principal:** GET/POST/PUT /api/admin/horarios -> valida día/rol/hora -> escribe grupo_horario.

5.2 Catálogo

CU-O-06 — Consultar catálogo

- **Actor:** Publico / Cliente
- **Descripcion:** Listar y filtrar productos, ver detalle, categorias y marcas; registrar eventos.
- **Flujo principal:** GET /api/catalogo/productos/{id}|categorias|marcas; POST /api/catalogo/eventos.

CU-O-07 — Gestionar categorias

- **Actor:** Administrador
- **Descripcion:** Crear, editar y activar/desactivar categorias.
- **Flujo principal:** GET/POST/PUT/PATCH /api/admin/catalogo/categorias.

CU-O-08 — Gestionar marcas

- **Actor:** Administrador
- **Descripcion:** Crear, editar y activar/desactivar marcas.
- **Flujo principal:** GET/POST/PUT/PATCH /api/admin/catalogo/marcas.

CU-O-09 — Gestionar productos

- **Actor:** Administrador
- **Descripcion:** Crear, editar, consultar y activar/desactivar productos y su categorizacion.
- **Flujo principal:** GET/POST/PUT/PATCH /api/admin/catalogo/productos.

CU-O-10 — Gestionar variantes y atributos

- **Actor:** Administrador
- **Descripcion:** Definir variantes (SKU, precio, costo), atributos, valores y asociaciones.
- **Flujo principal:** POST/PUT/PATCH /api/admin/catalogo/variantes, /atributos, /variantes/{id}/atributos.

5.3 Proveedores y Compras

CU-O-11 — Emitir orden de compra

- **Actor:** Encargado de Compras
- **Descripcion:** Crear una orden a un proveedor con items y bodega destino.
- **Flujo principal:** POST /api/compras/ordenes -> valida items -> numero OC -> impuesto por item -> totales por trigger.

CU-O-12 — Consultar ordenes de compra

- **Actor:** Compras / Gerente
- **Descripcion:** Listar ordenes y ver el detalle con cantidades recibidas.
- **Flujo principal:** GET /api/compras/ordenes, GET /api/compras/ordenes/{id}.

CU-O-13 — Registrar recepcion de mercancia

- **Actor:** Encargado de Bodega
- **Descripcion:** Recibir (total o parcial) los items de una orden, subiendo stock.
- **Flujo principal:** POST /api/compras/ordenes/{id}/recepciones -> valida pendientes -> kardex entrada_compra -> actualiza estado de la orden.

CU-O-14 — Registrar pago a proveedor

- **Actor:** Compras / Gerente
- **Descripcion:** Liquidar (total o parcial) una cuenta por pagar.
- **Flujo principal:** GET /api/compras/cuentas-por-pagar; POST /cuentas-por-pagar/{id}/pagos -> valida saldo -> actualiza estados.

5.4 Inventario y Bodegas

CU-O-15 — Transferir stock entre bodegas

- **Actor:** Encargado de Bodega
- **Descripcion:** Mover unidades de una variante de una bodega a otra.
- **Flujo principal:** POST /api/inventario/transferencias -> salida_transferencia (origen) + entrada_transferencia (destino) con kardex.

CU-O-16 — Consultar transferencias

- **Actor:** Bodega / Gerente
- **Descripcion:** Listar el historial de transferencias entre bodegas.
- **Flujo principal:** GET /api/inventario/transferencias.

CU-O-17 — Consultar existencias

- **Actor:** Roles operativos
- **Descripcion:** Ver el stock disponible por variante y bodega.
- **Flujo principal:** GET /api/referencias/stock?varianteld&bodegald.

5.5 Ventas y Pedidos

CU-O-18 — Realizar pedido

- **Actor:** Vendedor
- **Descripcion:** Registrar un pedido con su detalle y descontar stock.
- **Flujo principal:** POST /api/ventas/pedidos -> snapshot de precio -> salida_venta -> estado 'confirmado'.

CU-O-19 — Consultar pedidos

- **Actor:** Vendedor / Administrador
- **Descripcion:** Listar pedidos operativos y su detalle; consulta global de admin.
- **Flujo principal:** GET /api/ventas/pedidos/{id}; GET /api/pedidos/admin/todos.

CU-O-20 — Consultar mis pedidos

- **Actor:** Cliente
- **Descripcion:** Ver unicamente los pedidos propios (aislados por RLS).
- **Flujo principal:** GET /api/pedidos/{userId} (app.cliente_id via RLS).

5.6 Facturacion

CU-O-21 — Registrar factura de compra

- **Actor:** Encargado de Compras
- **Descripcion:** Facturar una orden y abrir la cuenta por pagar.
- **Flujo principal:** POST /api/compras/ordenes/{id}/facturas -> copia detalle -> cuenta_por_pagar; GET /facturas/{id}.

CU-O-22 — Emitir factura de venta

- **Actor:** Vendedor
- **Descripcion:** Facturar un pedido con snapshot de datos fiscales.
- **Flujo principal:** POST /api/ventas/pedidos/{id}/factura -> numero FV -> pedido a 'pagado'; GET /facturas/{id}.

CU-O-23 — Generar PDF de factura

- **Actor:** Vendedor / Compras
- **Descripcion:** Descargar el comprobante imprimible (compra o venta).
- **Flujo principal:** GET /api/ventas/facturas/{id}/pdf, GET /api/compras/facturas/{id}/pdf (iText).

5.7 Logistica y Envios

CU-O-24 — Despachar pedido

- **Actor:** Encargado de Despacho
- **Descripcion:** Generar el envio con guia y detalle, avanzando el pedido a 'despachado'.
- **Flujo principal:** POST /api/ventas/pedidos/{id}/despacho -> envio + envio_detalle + seguimiento_envio.

CU-O-25 — Consultar seguimiento de envio

- **Actor:** Despacho / Cliente
- **Descripcion:** Ver el rastreo del envio y el detalle del mismo.
- **Flujo principal:** GET /api/ventas/envios/{id}, GET /api/ventas/envios/{id}/seguimiento.

5.8 Devoluciones

CU-O-26 — Procesar devolucion

- **Actor:** Vendedor / Despacho
- **Descripcion:** Registrar la devolucion (RMA), reingresar stock y dejar el pedido 'devuelto'.
- **Flujo principal:** POST /api/ventas/pedidos/{id}/devolucion -> valida motivo/cantidades -> entrada_devolucion_cliente.

CU-O-27 — Consultar devolucion

- **Actor:** Roles operativos
- **Descripcion:** Ver la cabecera y el detalle de una devolucion.
- **Flujo principal:** GET /api/ventas/devoluciones/{id}.

6. Matriz de trazabilidad consolidada

Relaciona cada paquete operativo con su objetivo, los casos de uso que lo realizan y los requisitos funcionales principales que los soportan.

Paquete	Objetivo	Casos de uso	RF principales
Seguridad y Control de Acceso	Autenticar, autorizar y restringir por horario.	CU-O-01..05	RF-SEG-01..07
Catalogo	Mantener y publicar el catalogo comercial.	CU-O-06..10	RF-CAT-01..08
Proveedores y Compras	Abastecer y pagar a proveedores.	CU-O-11..14	RF-CMP-01..06
Inventario y Bodegas	Controlar existencias y movimientos.	CU-O-15..17	RF-INV-01..04
Ventas y Pedidos	Registrar pedidos y su ciclo.	CU-O-18..20	RF-VEN-01..06
Facturacion	Emitir facturas (compra/venta) y PDF.	CU-O-21..23	RF-FAC-01..05
Logistica y Envios	Despachar y dar seguimiento.	CU-O-24..25	RF-LOG-01..04
Devoluciones	Procesar devoluciones y reingresar stock.	CU-O-26..27	RF-DEV-01..04

7. Hallazgos y deuda tecnica

Pendientes reales detectados en el sistema actual durante la lectura de BD, backend y frontend. No provienen de versiones anteriores del producto.

- **DT-01:** Numeracion de folios no fiscal: los documentos (OC/RM/FC/PED/FV/EN/DV) usan el patron PREFIJO-YYYYMMDD-##### con sufijo aleatorio; la unicidad la respalda una restriccion UNIQUE, pero no es un consecutivo fiscal ni una serie por documento.
- **DT-02:** Transferencia sin detalle: transferencia_bodega es solo cabecera; la variante y la cantidad viven en los movimientos del kardex y en un texto de observacion. Falta una tabla de lineas para transferencias multi-item.
- **DT-03:** app.cliente_id autodeclarado: el aspecto fija SET LOCAL app.cliente_id a partir de principal.getClientId() (valor que viaja en el JWT). La RLS del cliente confia en ese valor de sesion en lugar de derivarlo en la BD desde el usuario autenticado.
- **DT-04:** Ajuste de inventario sin endpoint: existen la tabla ajuste_inventario y los tipos entrada_ajuste/salida_ajuste, pero no hay caso de uso expuesto para conteos/ajustes fisicos.
- **DT-05:** Kardex sin lectura via API: movimiento_inventario se escribe en cada operacion, pero no hay endpoint para consultarlo; solo se expone el stock actual (/api/referencias/stock).
- **DT-06:** Venta sin reserva de stock: el pedido descuenta stock directo (salida_venta) al confirmarse; la tabla reserva_stock existe pero no participa del flujo (decision documentada, no un defecto, pero limita escenarios de reserva).
- **DT-07:** Facturacion electronica pendiente: factura_venta.clave_acceso existe pero no se genera; no hay firma ni integracion con la autoridad fiscal (por confirmar el alcance previsto).
- **DT-08:** Bloqueo por intentos fallidos: usuario.intentos_fallidos y bloqueado_hasta existen, pero la aplicacion efectiva del bloqueo en el login queda por confirmar.
- **DT-09:** Actualizacion de estados de envio: tras el despacho ('en_transito') no hay endpoint para avanzar a 'entregado' o registrar incidencias de seguimiento (por confirmar).
- **DT-10:** Modulos legacy conviven con el operativo: carrito, wishlist, recomendaciones y analytics (ClickHouse) siguen en el codigo, pero no forman parte del nucleo operativo transaccional documentado aqui.

Nota de verificacion: los datos de esquema (102 tablas, 8 roles grp_, 34 tablas con RLS, 43 politicas, funciones esta_en_horario/fn_*), los endpoints y las reglas de negocio se obtuvieron leyendo directamente la BD PostgreSQL y el codigo fuente. Los puntos marcados como 'por confirmar' no pudieron verificarse de forma concluyente con el material disponible.*